

Purchase Module Workflow — How It Actually Works

Written: 2026-10-03, by Claude, from direct reading of the current backend code (`purchases.service.ts` , `purchase-requests.service.ts` , `purchase-rules.ts` , `purchase-request-rules.ts` , `purchase.dto.ts` , `purchase-request.dto.ts` , `schema.prisma`), not from memory or the historical archive. Everything below is VERIFIED against the live code as of this date, including the same-day rule-11 refactor.

This exists because comparing DMS against `AI_reports/Purchase_Module_Design_Report.md` (a generic multi-document procure-to-pay blueprint) raised real doubts about how *this* system's Purchases module actually behaves. This document answers that directly: not “how should it work,” but “how does it work, right now, in this code.”

1. The core idea: two documents, loosely linked

Unlike the AI report's model (Requisition → RFQ → PO → GRN → Invoice, each a separate matched document), DMS has **two** document types, and the second does most of the work:

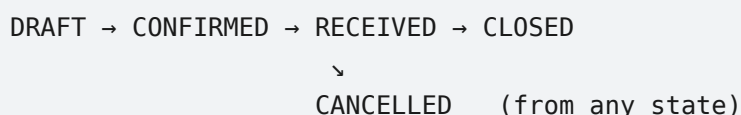
- **PurchaseRequest** — optional. An internal “we need to buy this” record. Nothing legally binding, no external party involved.
- **Purchase** — the actual transaction record. This single record plays the role the AI report splits across PO + GRN + Invoice + Payment: it holds the line items, the payments, the documents (invoice scans etc.), and a status that covers both “has it arrived” and “is it closed.”

A Purchase does **not** need a PurchaseRequest behind it — most purchases in the system have none (`purchaseRequestId` is nullable). When one exists, the link happens at two levels:

- **Header level:** `Purchase.purchaseRequestId` — “this purchase was raised to fulfill this request” (informational, shown on both detail pages).
- **Line level:** `PurchaseItem.purchaseRequestId` — “this specific line fulfills this specific requested line.” This is the one that actually drives the fulfillment math (\$3).

2. Status lifecycles

2.1 Purchase.status



Important: nothing in the code enforces this diagram. `status` is a plain field on the update form (`purchase.dto.ts`) — any user with `purchases.edit` can set a Purchase to any of the five values at

any time, including jumping straight from DRAFT to CLOSED, or back from CLOSED to DRAFT. The arrows above describe the *intended* flow a user would normally follow, not a rule the backend checks.

Nothing auto-closes a Purchase. A purchase that's RECEIVED and fully PAID stays RECEIVED forever unless someone manually sets it to CLOSED. The dashboard's "open items" list reflects this honestly — it's why a fully-paid purchase can still show up as "open."

2.2 Purchase.paymentStatus

UNPAID → PARTIAL → PAID

This one you genuinely cannot set by hand — it's not in the update DTO at all. It's recomputed every time `create`, `update`, or `addPayment / removePayment` runs, from one function:

```
function derivePaymentStatus(totalAmount, paidAmount) {
  if (paidAmount <= 0) return 'UNPAID';
  if (paidAmount >= totalAmount) return 'PAID';
  return 'PARTIAL';
}
```

`totalAmount` is the sum of line `totalPrice` values (not quantity × unit price — see §5). `paidAmount` is the sum of that purchase's `PurchasePayment` rows. Both are **stored**, not computed live on every read — this is deliberate (fast list views), which is also why they can never be written directly from anywhere else.

2.3 PurchaseRequest.status

DRAFT → SUBMITTED → APPROVED → PARTIALLY_PURCHASED → COMPLETED



REJECTED



(back to APPROVED if a
linked purchase is cancelled/reduced)

CANCELLED

Two very different mechanisms touch this field, and mixing them up is probably the single biggest source of confusion:

(a) Manual edit — same pattern as `Purchase.status`: the update form has a free status dropdown, no transition rules enforced. A user can set a request straight to `COMPLETED` by hand even if nothing was ever purchased against it.

(b) Automatic recompute (`PurchaseRequestsService.recomputeStatus()`) — runs automatically whenever a linked `Purchase` is created, edited, or deleted. It **only** touches a request currently in `APPROVED`, `PARTIALLY_PURCHASED`, or `COMPLETED` (`AUTO_MANAGED_STATUSES`) — a request sitting in `DRAFT`, `SUBMITTED`, `REJECTED`, or `CANCELLED` is never touched by it. For an eligible request, it:

1. Sums actual purchased quantity per requested line (via `PurchaseQuantitiesService`, see §3) — **cancelled purchases don't count.**

2. If nothing purchased yet → APPROVED. If every line fully covered → COMPLETED. Otherwise → PARTIALLY_PURCHASED.
3. Writes the new status **only if it changed**, with an audit log entry noting it was automatic.

The practical trap: because (a) and (b) both write the same field, a user can manually force a request to COMPLETED, and then the *next* time a linked purchase changes, the automatic recompute can silently move it right back to PARTIALLY_PURCHASED or APPROVED — because the recompute doesn't know or care that a human set it on purpose. There's no “lock” distinguishing a manual status from a system-computed one.

3. How “how much has actually been purchased” is calculated

As of the same-day refactor, there is exactly **one** place this is computed: `PurchasesService`'s `PurchaseQuantitiesService.sumQuantitiesByRequestItem(requestItemIds)`. It sums `PurchaseItem.quantity` grouped by `purchaseRequestItemId`, **excluding** any `PurchaseItem` whose parent `Purchase.status = CANCELLED`.

Two callers use this, and only this:

- `PurchaseRequestsService.withItemQuantities()` — powers the “quantity purchased / quantity remaining” columns on the request detail page.
- `PurchaseRequestsService.recomputeStatus()` — decides APPROVED/PARTIALLY_PURCHASED/COMPLETED (§2.3b).

Before the refactor, each of these had its **own** copy of this logic (plus a third copy sitting inside `recomputeStatus()` itself) — three places that had to agree on “what counts as purchased,” which is exactly the kind of thing that drifts silently. Now there's one.

What “purchased” does *not* account for: whether the purchase has actually arrived (RECEIVED), only that a `PurchaseItem` row referencing that request-item exists on a non-cancelled `Purchase`. A DRAFT purchase that references a request item counts as “purchased” for this calculation the moment it's created — there's no separate “ordered vs. received” distinction the way the AI report's PO/GRN split has. This is the single clearest place where DMS's one-document model loses information the two-document model would have kept.

4. Payments, Documents, Returns — what each does and doesn't touch

- **PurchasePayment** — add/remove freely (`purchases.manage`). Every add/remove triggers `recomputePaymentTotals()`, which recalculates `paidAmount` and `paymentStatus` on the parent `Purchase`. Never touches `PurchaseRequest`.
- **PurchaseDocument** — file attachments (invoice scans, etc.), gated behind its own permission (`documents.upload`), separate from `purchases.manage`. Purely informational, doesn't affect any derived field.
- **PurchaseReturn** (Return-to-Vendor) — records what went back to the supplier and its credit value. **Deliberately does not touch totalAmount/paidAmount/paymentStatus** — those represent what

was originally billed, and changing them would erase the original transaction's history. **Known, accepted limitation:** a `PurchaseReturnItem` has a `Restrict` foreign key to the `PurchaseItem` it returned against, and `PurchasesService.update()` deletes-and-recreates all line items on every edit — so **a purchase with any return on it cannot be edited at all**, not even a status change. This was flagged previously and accepted as a trade-off rather than reworking the core update logic.

None of these three ever touch Inventory — there is no Inventory module yet, and CLAUDE.md hard rule 7 explicitly forbids Purchases from triggering stock movements until a future business requirement says otherwise.

5. Money fields: what's derived vs. what's typed in

Field	Who sets it	Notes
<code>PurchaseItem.unitPrice</code>	User, optional	Lump-sum/exception pricing allowed — not every line has a meaningful per-unit price.
<code>PurchaseItem.totalPrice</code>	User, always required	Never validated against $\text{quantity} \times \text{unitPrice}$ — this is a deliberate, explicit business rule (original spec), not a gap. The frontend auto-fills a suggestion when qty/price change, but the field stays directly editable and the backend never checks the arithmetic.
<code>Purchase.totalAmount</code>	System	Sum of all <code>PurchaseItem.totalPrice</code> . Recomputed on every create/update.
<code>Purchase.paidAmount</code>	System	Sum of all <code>PurchasePayment.amount</code> . Recomputed on every payment add/remove.
<code>Purchase.paymentStatus</code>	System	See §2.2.

6. Permissions — one notable surprise

Action	Permission required
View/list Purchases	<code>purchases.view</code>
Create a Purchase	<code>purchases.manage</code>
Edit a Purchase	<code>purchases.edit</code>
Delete a Purchase, add/remove payments or returns	<code>purchases.manage</code>
View/list Purchase Requests	<code>purchases.view</code>
Create a Purchase Request	<code>purchases.manage</code>

Edit a Purchase Request	<code>purchases.edit</code>
Upload/delete a document	<code>documents.upload</code> (separate from everything else)

There is no separate `purchase-requests.*` permission set — Purchase Requests piggybacks entirely on the Purchases permissions. Anyone who can manage Purchases can manage Purchase Requests, and there's no way to grant one without the other. If you ever want “can raise a request but not finalize a purchase” as a distinct role capability, that doesn't exist today and would need new permissions added to `PERMISSION_CATALOG`.

7. What's deliberately *not* here (compare against [AI_reports/Purchase_Module_Design_Report.md](#))

- No RFQ / supplier quote comparison — a Purchase is entered directly, no sourcing stage.
- No separate Goods Receipt document — RECEIVED is just one value of Purchase.status, with no quantity-received, receiver-identity, or condition-notes record behind it.
- No Quality Inspection / QC-hold concept.
- No three-way match (PO vs. receipt vs. invoice) — there's only one document, so there's nothing to match against.
- No tiered/value-based approval workflow or segregation-of-duties enforcement — status is a flat field anyone with the right permission can set to anything (§2).
- No automatic PO closure, no GRNI accrual, no landed cost.

All of these are real gaps *relative to a full manufacturing-grade procurement system* — but every one of them is either an explicit, deliberate business rule already on record (e.g. “don't force Total Price = Qty × Unit Price,” “Purchases don't touch Inventory”) or simply hasn't been requested. None of this should be built speculatively — see CLAUDE.md rule 9 and the architecture rule about not inventing business logic.

8. Quick answers to likely doubts

- **“Why does a cancelled purchase disappear from the request's purchased quantity?”** — Deliberate. `COUNTABLE_PURCHASE_WHERE` / the quantities service exclude CANCELLED everywhere, because a cancelled purchase never delivered anything.
- **“Why does this fully-paid, received purchase still show as 'open' on the dashboard?”** — Because nothing auto-closes a Purchase; CLOSED is a manual action nobody took yet. Honest reflection of real status, not a bug.
- **“I set a request to COMPLETED by hand, why did it change back?”** — The automatic recompute ran again (triggered by an edit to a linked purchase) and didn't know your manual edit was intentional. There's no lock preventing this.
- **“Can a Purchase Request be rejected after being approved?”** — Nothing stops you from setting any status to any other status manually; there's no enforced state machine for user-driven edits,

only for the automatic recompute's narrow

APPROVED ↔ PARTIALLY_PURCHASED ↔ COMPLETED cycle.

- **“Why can't I edit this purchase?”** — Check whether it has a return recorded against it (\$4) — that's the one hard block.